

ARL Rover Safety Monitor Debugging Report



Name: Natalie Sameh Saad

Code: 2500617

Project: ARL Rover Safety Debugging System

Date: 16/08/2026

System Process Flow:(from input to braking decision)

- 1- Validating all detection variables as (forward, left and confidence) are valid and finite values.
- 2- Starting obstacle detection and converting heading angles from degrees to radian as it is the angle between forward component and X_{world} .
- 3- Detect each obstacle and get the range then check that the confidence and range are in the valid range.
- 4- Extracting the valid obstacles Data including (ID, forward and left positions).
- 5- Converting the local position of obstacle to world position by resolution and using radian heading angles.
- 6- Checking if the obstacles list is empty, it returns a null value, otherwise, proceeding to find nearest obstacle with shortest range.
- 7- Calculating stopping distance depending on its speed after conversion from Km/hr to m/sec. and adding reaction distance and braking distance.
- 8- Emergency Brake must engage when at least one valid obstacle is in front of rover , is inside the lane corridor, has a forward distance shorter than the stopping distance).

Known Issues and Bugs:

1- Missing Source File in CmakeLists.txt:

- **Symptom:** Compilation error occurred indicating that functions cannot be found.
- **Root Causes:** The source file (safety_monitor.cpp) was not added to the add_executable command in the CmakeLists.txt file.
- **Fix Applied:** Adding (safety_monitor.cpp) to add_executable command which already includes main.cpp
- **Reasoning and Programming Concept:** Cmake must include all source files in the build configuration, otherwise, it cannot link the files together so the compiler fails in locating functions defined in not included files.

2- Undefined Optional Type for obstacle Detection:

- **Symptom:** The std::optional wasn't defined so it caused errors in safety_monitor.cpp and safety_monitor_hpp.
- **Root Causes:** As the optional library wasn't added in both files.
- **Fix Applied:** adding of #include optional in both files to add the library.
- **Reasoning and Programming Concept:** Libraries must be defined in the code and using optional is important to specify whether an obstacle has been detected or not .

3- Incorrect unit conversion in angles(degrees to radians):

- **Symptom:** Obstacle position was wrong.
- **Root Causes:** The absence of the conversion formula from degrees to rad which caused errors.
- **Fix Applied:** Defining of PI and added the conversion formula by multiplying the angle by PI and dividing by 180.

- **Reasoning and Programming Concept:** cmath library functions expect angles to be in rad not degrees to get accurate calculations.

4- Sign Error in obstacle Coordinate (local to world):

- **Symptom:** Obstacle coordinates were incorrect on the map.
- **Root Causes:** In World transformation formula for X_{world} equals $(pose.worldX + \cos(\theta) * detection.forward - \sin(\theta) * detection.left)$ and it was written +ve not -ve which is wrong.
- **Fix Applied:** Correcting +ve sign with -ve sign before sine in X_{world} .
- **Reasoning and Programming Concept:** In resolution of left in direction of X, it makes cosine ($\theta + 90$) which is $-\sin(\theta)$ to get accurate coordinates.

5- Index off-by-one Error in loop:

- **Symptom:** The loop escaped obstacles by showing 1 instead of 3.
- **Root Causes:** Due to the presence of (+1) in the loop index so it skip valid obstacles.
- **Fix Applied:** Removing (+1) from the loop to get the correct numbr of obstacles.
- **Reasoning and Programming Concept:** As the index start from zero and reaching the last number but `detections.size()` is this last number (+1) due to counting the zero and in case of presence of index +1 is less than that size in the loop , it won't detect the last one as one will be added to it and become equal to the size not less than so (+1) should be removed to become correct and detect all.

6- Confidence incorrect comparison:

- **Symptom:** Detection of confidence is less than minimum confidence.
- **Root Causes:** Detection become weak as it was lower than minimum.
- **Fix Applied:** changing less than to bigger than minimum value and to 1.

- **Reasoning and Programming Concept:** After changing all the valid obstacles that are selected will be truly valid with high confidence not reversed.

7- Logic error in Nearest Obstacle Selection:

- **Symptom:** choosing wrong obstacle with high range to be the nearest.
- **Root Causes:** The presence of ($>$ nearest range) leads to choosing the larger range not the smaller.
- **Fix Applied:** Changing of greater than to less than the nearest range.
- **Reasoning and Programming Concept:** After changing the logic will choose the obstacle with smaller range than the previous one range until it reaches the smallest one.

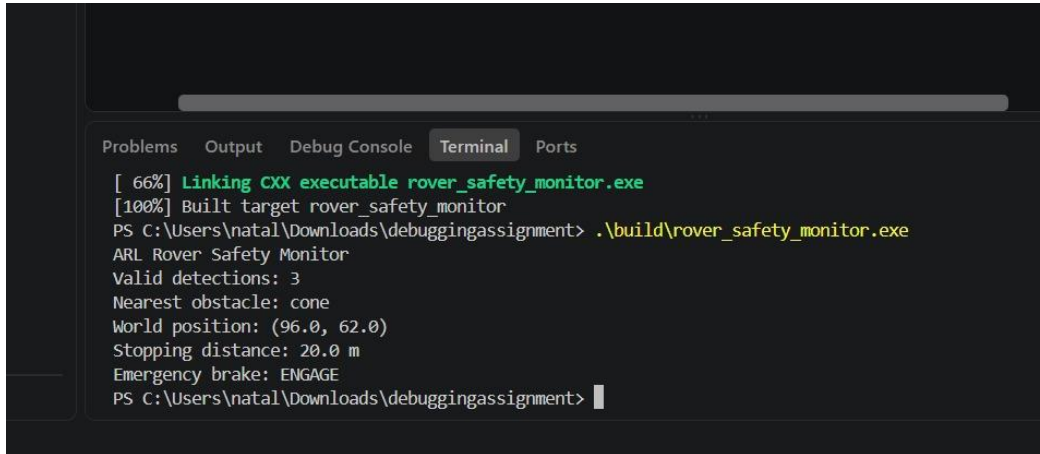
8- Kinematic Unit Conversion Error of speed:

- **Symptom:** Stopping distances were wrong and the unit of speed conflicted with other units.
- **Root Causes:** Not using (m/sec) but Km/hr which is wrong.
- **Fix Applied:** Converting from Km/hr to m/sec by $(\times 1000.0/3600.0)$
- **Reasoning and Programming Concept:** Speed unit should be matching SI units like maximumDeceleration is m/sec^2 , stopping distance in metres and reactiontime in seconds.

9- Error in Emergency Brake logic:

- **Symptom:** always clear not affected by the certain conditions.
- **Root Causes:** code was not complete to check for these conditions to become engage or clear.
- **Fix Applied:** Adding Code of if statement for conditions.
- **Reasoning and Programming Concept:** After adding the code logic will detect the conditions as soon as conditions occur with obstacle the emergency brake will be engaged.

Build and Execution Verification:



The screenshot shows a Visual Studio Code terminal window with the 'Terminal' tab selected. The terminal output displays the progress of building and running a program named 'rover_safety_monitor.exe'. The build process is shown as 100% complete. The execution output shows the program running successfully, displaying simulation data for an ARL Rover Safety Monitor.

```
Problems Output Debug Console Terminal Ports
[ 66%] Linking CXX executable rover_safety_monitor.exe
[100%] Built target rover_safety_monitor
PS C:\Users\natal\Downloads\debuggingassignment> .\build\rover_safety_monitor.exe
ARL Rover Safety Monitor
Valid detections: 3
Nearest obstacle: cone
World position: (96.0, 62.0)
Stopping distance: 20.0 m
Emergency brake: ENGAGE
PS C:\Users\natal\Downloads\debuggingassignment> |
```

References and Resources:

- Cmake Documentations.
- Visual studio code with c++ extension.
- Gemini Ai (helped me to set cmake up in vs code due to errors).